

Optimal control in SMC

Marko Guberina

September 24, 2025

0.1 Optimal control

We use the Box-FDDP solver [reference] from the Crocoddyl optimal control suite [reference]. It's a DDP-style trajectory optimizing method. It works on the dynamics level. Since we only do velocity control, we use the velocity obtained from the forward rollout as the actual control input.

Crocoddyl's main benefits are that it is (very) fast, and that it uses Pinocchio for it's dynamics computations (same as the rest of the codebase). It's main drawback is the verbosity of the API, and the difficulty of defining novel costs or constraints. This is because Crocoddyl is written in C++, and just pybind's to expose it's API in Python. Meaning, if something is missing, it has to be written in C++, and then exported to Python. While this isn't prohibitively difficult, it entails compiling the library (and likely it's dependencies) and requires a working knowledge of C++, Pybind and the library design. In other words, this should be pursued only if there is a good reason to do so as the required implementation time will detract from other tasks.

A generic optimal control problem looks like

$$X^*, U^* = \operatorname{argmin}_{X, U} \sum_{k=1}^{N-1} \int_{t_k}^{t_{k+1}} l(x, u) dt + L(x(N)) \quad (1)$$

$$\text{such that } \dot{x} = f(x, u) \quad (2)$$

$$x \in \mathcal{X}, u \in \mathcal{U} \quad (3)$$

$$(4)$$

A common choice for $\Delta t = t_{k+1} - t_k$ is $\Delta t = 0.01$.

Since we are working exclusively with trajectory optimization, we can make use of the additional structure of the solver. Box-FDDP is a direct multiple-shooting trajectory optimization method. Multiple-shooting means that states x_i are also decision variables. Trajectory optimization means that we work with a candidate solution, or a solution guess which is iteratively improved. One iteration consists of:

1. A forward simulation of the dynamics with the current control input candidate, which includes the costs. This is referred to as the forward pass.

2. Linearising the dynamics and the costs around every point the obtained trajectory.
3. Updating the candidate solution by performing a line search in the direction of most decreasing cost. This is referred to as the backward pass.

The software structure of this is a list of N `actionModel`s, each of which contains a dynamics model and a cost model. Since the cost is in the end a scalar, this means that these action models do not need to be the same — the dynamics, cost, linearization and line search update computations are performed at each action model (also referred to as knot or node) individually. We use this fact in the path-following formulation, more on that later. Most importantly, we add a much larger goal cost on the last node, called the terminal node.

Depending on the task, we create a different objective function. Before doing that, we introduce the specific generic parts we always have, and a short note on how to go from optimal control for planning to real-time MPC.

0.1.1 Turning OCPs into MPCs

Any OCP described here is turned into an MPC by:

- **Shortening the time horizon N , capping the maximum number of solver iterations $n_{\text{solver_iter}}$, and reducing control frequency f_{ctrl}** until satisfactory performance is reached. From limited qualitative testing, the most common numbers are $N = 30$, $n_{\text{solver_iter}} = 10$, and $f_{\text{ctrl}} = 200$. In code this is done simply by passing arguments.
- **Warmstarting** with the previous solution. The previous solution is offset by the amount of time passed. TODO: make sure it is, make sure the `ctrl_freq` amount of time, not Δt time. If the second is the case, you probably want to linearly interpolate to offset correctly - also make sure you're time-testing whether this is actually helpful (if it doesn't shave off an iteration off of the solver, it isn't)
- don't have zero velocity in the terminal node

In the case that the decreased control frequency causes problems, some kind of low-pass or similar filtering can be applied to the incoming control signals, smoothening them out, but this hasn't been done.

0.1.2 Generic OCP components

Every OCP we construct has these components.

Decision variables, dynamics constraint In our case, the state is a vector of joint positions and velocities:

$$x = [q \quad v]^T. \quad (5)$$

The control inputs can be either acceleration or torques, depending on which we formulate either the inverse or the forward dynamics problem respectively, and solve for the other quantity. We used inverse dynamics. In any case, because the states are also decision variables, a state-input pair must follow the system dynamics into the next that. Thus to put in the discrete version of $\dot{x} = f(x, u)$, we can write

$$\text{dynamics}(\delta x_{k+1}, \delta x_k, \delta u_k) = 0 \quad (6)$$

$$\text{dynamics}(\delta \dot{x}, \delta x, \delta u) = \delta \dot{x} - (f_x \delta x + f_u \delta u) \quad (7)$$

State constraints Since we use Box-FDDP, we can only have box constraints on torque inputs, i.e. $\tau_{\min} \leq \tau \leq \tau_{\max}$. In the case of an underactuated joint, we set the torque limit on it to zero.

To put constraints on the state, namely on arms' joint limits and on all joints' velocities, we put them into a quadratic barrier function which we put into the objective function. In particular (TODO make sure this is correct):

$$r_{lb} = x - x_{\min} \quad (8)$$

$$r_{ub} = x - x_{\max} \quad (9)$$

$$B(x) = \begin{cases} \frac{1}{2}(\|r_{lb}\|^2 + \|r_{ub}\|^2) & \text{if } lb \leq x \leq ub \\ \text{max float} & \text{otherwise} \end{cases} \quad (10)$$

Regulation costs (regularization) Finally, we add regulation costs on both the state $c_1 x^T x$, and control inputs $c_2 u^T u$.

0.1.3 Point to point

In case there is a fixed end-goal to reach, we use the $se(3)$ error norm as the task-specific function to put into the objective function.

Single arm end-effector reference TODO In the particular case of the goal being defined for the end-effector, whose pose is denoted as T_w^e reaching a goal denoted as T_w^{goal} :

$$l_e(x) = \left\| \log \left((T_w^e)^{-1} \right) T_w^{\text{goal}} \right\|_2 \quad (11)$$

Here we of course use only joint values q , not the full state x , because the end-effector pose is only a function of q .

We put this cost function at every knot. This method works fine.

How to run The main is in `examples/optimal_control/point_to_point/croco_ee_reference_p2p_mpc.`

Remember to set `-n-knots` to something small like 30, and `-max-solver-iter` to something like 10 so that this runs in reasonable time!

Base and single arm end-effector reference In addition to the end-effector distance to goal cost, we add the same one for the base. Despite the base being an $SE(2)$ joint, we use an $SE(3)$ error as well. This is simply because this is easier to do within the Crocoddyl framework, and it of course doesn't change anything. Thus the objective is given by:

$$l_e(x) = \left\| \log \left((T_w^e)^{-1} \right) T_w^{\text{ee_goal}} \right\|_2 + \left\| \log \left((T_w^b)^{-1} \right) T_w^{\text{base_goal}} \right\|_2 \quad (12)$$

Alternatively, we can define the base cost to be defined only through position (i.e. have no orientation cost). The objective is then

$$l_e(x) = \left\| \log \left((T_w^e)^{-1} \right) T_w^{\text{ee_goal}} \right\|_2 + \|p_{\text{base_goal}} - p_{\text{base}}\|_2 \quad (13)$$

We put this cost function at every knot. This method works fine for the most part, but has trouble converging right at the end. It looks as if it continues to actuate the base to move the end-effector, despite the base being already at the goal. This is particularly the case with a position-only cost in the base. A simple hack would be to let a Cartesian control end the control loop after the overall error is small enough.

How to run The main is in `examples/optimal_control/point_to_point/croco_base_and_ee_reference`. Remember to set `-n-knots` to something small like 30, and `-max-solver-iter` to something like 10 so that this runs in reasonable time!

Dual arm end-effector reference The objective is simply the sum of left end-effector goal cost and right end-effector goal cost:

$$l_e(x) = \left\| \log \left((T_w^{el})^{-1} \right) T_w^{\text{l_ee_goal}} \right\|_2 + \left\| \log \left((T_w^{er})^{-1} \right) T_w^{\text{r_ee_goal}} \right\|_2 \quad (14)$$

Discussion It doesn't work because it can't converge — it gets pretty close, and then oscillates.

How to run The main is in `examples/optimal_control/point_to_point/croco_dual_ee_reference_p2p`. Remember to set `-n-knots` to something small like 30, and `-max-solver-iter` to something like 10 so that this runs in reasonable time!

Base and dual arm end-effector reference As expected, this is just a combination of the base and each end-effector costs:

$$l_e(x) = \left\| \log \left((T_w^{el})^{-1} \right) T_w^{\text{l_ee_goal}} \right\|_2 + \left\| \log \left((T_w^{er})^{-1} \right) T_w^{\text{r_ee_goal}} \right\|_2 + \|p_{\text{base_goal}} - p_{\text{base}}\|_2 \quad (15)$$

How to run The main is in `examples/optimal_control/point_to_point/croco_base_and_dual_ee_refe`. Remember to set `-n-knots` to something small like 30, and `-max-solver-iter` to something like 10 so that this runs in reasonable time!

Discussion Very bad performance. Not only does it not converge, but the arms move in a very strange way.

0.1.4 Trajectory following

The optimal control formulation allows us to optimize the time-law over the path. In other words, the total time for the path, or a velocity over it does not need to be imposed, but could instead be solved for within the optimal control problem as an additional decision variable. Let the prescribed path be parametrized by $s \in [0, 1]$. One simple approach would be to minimize the time taken to traverse the path by minimizing the time T which can be a function of $s \mapsto Ts \in [0, T]$. In fact this could be the only cost, while following the path can be considered as a constraint.

This has not been done in SMC due to development limited time. Instead, we assume a uniform velocity over the path to obtain a Cartesian trajectory. We then sample from this trajectory and set those points to be distance costs at corresponding action models. In other words, the OCP is identical to the point to point OCP, but instead of the error at every action model being towards the final goal, it's to the corresponding point on the trajectory. Despite this limitation, the path following versions work about as well as the point-to-point ones.

0.2 [MARKO TODO] Navigation (mobile base control)

[MARKO TODO] Following a path planner Always expect the initial point to be in the current position of the robot (our planner is fast)

[MARKO TODO] Following a prescribed path TODO: 1) introduce the function to get you to the closest point. 2) explain the need to smoothly interpolate between points (sometimes the given points are very sparse).